

Scale the Storage

As the autocomplete system grows, the trie becomes too large to fit into a single server.

Therefore, storage must be scaled horizontally using:

None

Sharding

Sharding divides trie data across multiple servers so the system can:

- Handle larger datasets
- Support more users
- Improve scalability
- Reduce server load

Why Scaling is Needed

As search traffic increases:

- Trie size grows continuously
- More prefixes are stored
- More top-k results are cached
- Memory requirements increase

A single server eventually becomes:

- Memory bottleneck
- CPU bottleneck
- Single point of failure

Thus, distributed storage is necessary.

Basic Sharding Strategy

Since only English lowercase characters are supported:

None

a-z

a simple idea is to shard based on:

None

First Character

This is called:

First-Level Sharding

Example: Two Shards

If we use:

- 2 servers

Data distribution can be:

Server	Character Range
Server 1	a – m
Server 2	n – z

Example: Three Shards

Server	Character Range
Server 1	a – i
Server 2	j – r
Server 3	s – z

Maximum First-Level Shards

Since English has:

None

26 letters

maximum first-level shards:

None

26 servers

Each server stores trie nodes beginning with one or more letters.

Problem with Naive Sharding

At first glance, first-character sharding appears reasonable.

However:

- Character distribution is not uniform.

Example:

- Many more words start with:

None

c

than:

None

x

This creates:

- Uneven storage distribution
- Uneven traffic load
- Hot shards

Example of Data Imbalance

Character	Query Volume
c	Very High
s	High
x	Very Low

z	Very Low
---	----------

Consequences:

- Some servers overloaded
- Others underutilized

This reduces:

- Scalability
- Efficiency
- Performance

Multi-Level Sharding

To support more than 26 servers:

- Further partition prefixes.

Example:

None

Queries starting with 'a'

can be divided into:

Shard	Range
Shard 1	aa – ag
Shard 2	ah – an

Shard 3	ao – au
Shard 4	av – az

This is called:

None

Second-Level Sharding

Additional levels can also be added if needed.

Smarter Sharding Approach

Naive alphabetical splitting still causes imbalance.

To solve this:

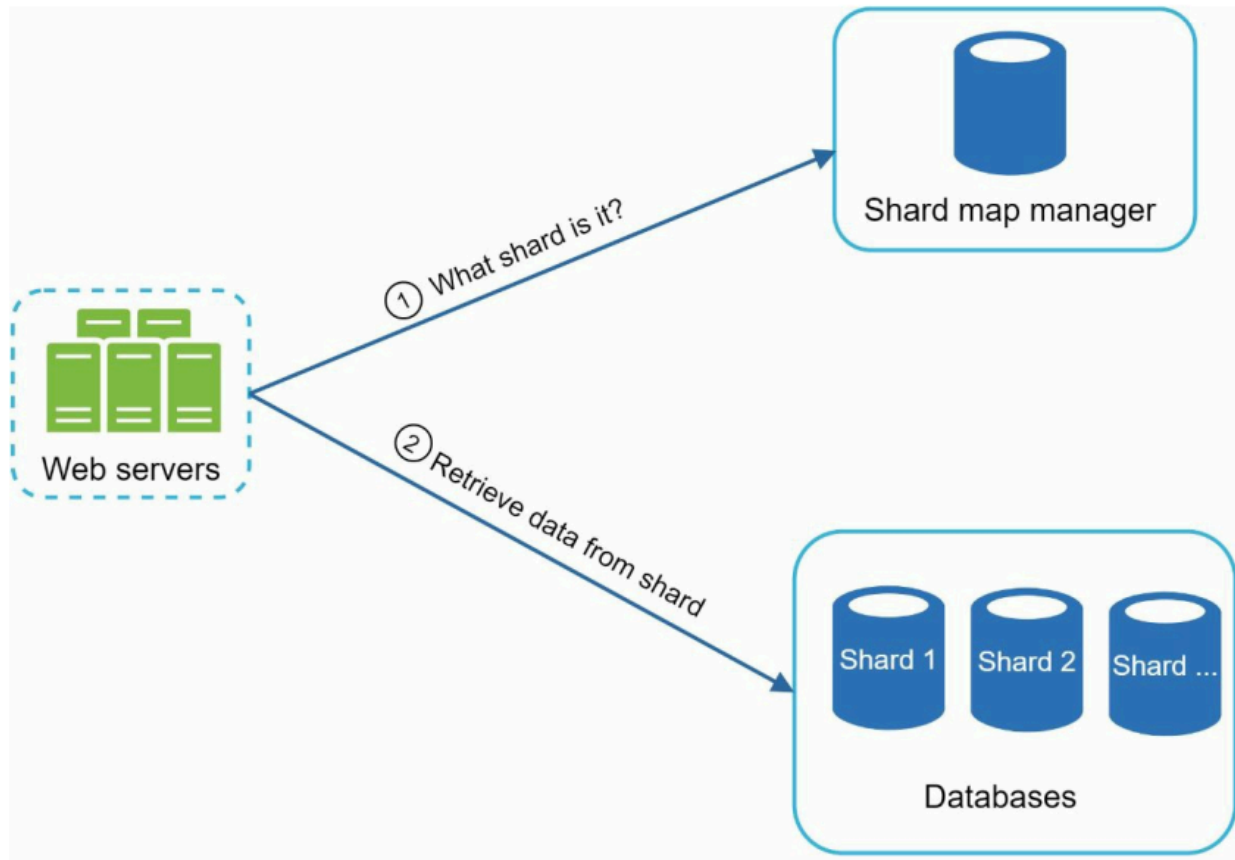
- Historical query distribution is analyzed.

The system uses:

None

Frequency-Based Sharding

instead of equal alphabet ranges.



This approach is shown in Figure.

Shard Map Manager

A special component called:

None

Shard Map Manager

maintains metadata about shard allocation.

Responsibilities:

- Track shard ranges
- Map prefixes to servers

- Balance storage distribution

The shard map manager stores:

None

`<prefix_range → shard_server>`

Example of Smart Sharding

Suppose historical analysis shows:

Prefix	Traffic Volume
s	Extremely High
u-z	Combined traffic similar to s

Then instead of:

None

`s-z on one server`

we use:

Shard	Range
Shard 1	s

Shard 2	u-z
---------	-----

This produces:

- Better load balancing
- More uniform traffic distribution

Benefits of Smart Sharding

Benefit	Description
Balanced load	Even query distribution
Better scalability	Easier expansion
Reduced hotspots	Prevent overloaded shards
Efficient memory use	Better server utilization

Query Routing Process

Step 1

User sends autocomplete request.

Example:

None

[search?q=sy](#)

Step 2

API server consults shard map manager.

Step 3

Shard manager identifies correct shard.

Step 4

Request routed to appropriate trie server.

Step 5

Top-k suggestions returned.

Dynamic Resharding

As traffic patterns evolve:

- Some shards may become overloaded.

The system can:

- Split hot shards
- Merge cold shards
- Rebalance data dynamically

This improves:

- Long-term scalability
- System stability

Complete Scalable Trie Architecture

Component	Responsibility
Trie Servers	Store shard data
Shard Map Manager	Prefix-to-shard mapping
API Servers	Route requests
Trie Cache	Fast lookup
Workers	Build shard data

Storage Scaling Techniques

Technique	Purpose
First-level sharding	Basic partitioning
Multi-level sharding	Support more servers
Frequency-based sharding	Balanced distribution

Dynamic resharding	Adaptive scaling
--------------------	------------------

Advantages of Distributed Trie Storage

Advantage	Description
Horizontal scalability	Add more servers easily
High availability	Failures isolated
Better performance	Reduced server load
Lower latency	Faster lookups

Final Design Goals Achieved

The scalable trie storage architecture supports:

- Massive datasets
- Millions of queries
- Balanced server utilization
- Efficient autocomplete retrieval
- Horizontal scaling

This enables the autocomplete system to remain:

- Fast
- Scalable

- Highly available
- Efficient under heavy traffic.